

## BootCamp\_04 – Sort Double Linked Lists (Optional)

### Student Information

**Integrity Policy:** All university integrity and class syllabus policies have been followed. I have neither given, nor received, nor have I tolerated others' use of unauthorized aid.

I understand and followed these policies:                      Yes                      No

Name:

Date:

### Submission Details

Final **Changelist** number:

Verified build:                      Yes                      No

(75% max) PA to be regraded:

Required Configurations:

Discussion (What did you learn):

## Verify Builds

- Follow the Piazza procedure on submission
  - Verify your submission compiles and works at the changelist number.
- Verify that only MINIMUM files are submitted
  - No – Generated files
    - \*.pdb, \*.suo, \*.sdf, \*.user, \*.obj, \*.exe, \*.log, \*.pdb, \*.db, \*.user
    - Anything that is generated by the compiler should not be included
  - No – Generated directories
    - /Debug, /Release, /Log, /ipch, /.vs
- Typical files project files that are required
  - \*.sln, \*.cpp, \*.h
  - \*.vcxproj, \*.vcxproj.filters, CleanMe.bat

## Standard Rules

### Submit multiple times to Perforce

- Submit your work as you go to perforce several times (at least 5)
  - As soon as you get something working, submit to perforce
  - Have reasonable check-in comments
    - Points will be deducted if minimum is not reached

### Write all programs in cross-platform C++

- Optimize for execution speed and robustness
- Working code doesn't mean full credit

### Submission Report

- Fill out the submission Report
  - No report, no grade

### Code and project needs to compile and run

- Make sure that your program compiles and runs
  - Warning level ALL ...
  - NO Warnings or ERRORS
    - Your code should be squeaky clean.
  - Code needs to work "as-is".
    - No modifications to files or deleting files necessary to compile or run.
  - All your code must compile from perforce with no modifications.
    - Otherwise it's a 0, no exceptions

### Project needs to run to completion

- If it crashes for any reason...
  - It will not be graded and you get a 0

### No Containers

- NO STL allowed {Vector, Lists, Sets, etc...}
  - No automatic containers or arrays
  - You need to do this the old fashion way - **YOU EARNED IT**

### Leave Project Settings

- Do NOT change the project or warning level
  - Any changing of level or suppression of warnings is an integrity issue

### Simple C++

- No modern C++
  - No Lambdas, Autos, templates, etc...
  - No Boost
- NO Streams
  - Used fopen, fread, fwrite...
- No code in MACROS
  - Code needs to be in cpp files to see and debug it easy
- **Exception:**
  - implicit problem needs templates

### Leaking Memory

- If the program leaks memory
  - There is a deduction of 20% of grade
- If a class creates an object using new/malloc
  - It is responsible for its deletion
- Any **MEMORY** dynamically allocated that isn't freed up is **LEAKING**
  - Leaking is **HORRIBLE**, so you lose points

### No Debug code or files disabled

- Make sure the program is returned to the original state
  - If you added debug code, please return to original state
- If you disabled file, you need to re-enable the files
  - All files must be active to get credit.
  - Better to lose points for unit tests than to disable and lose all points

### No Adding files to this project

- This project will work "as-is" do not add files...
- Grading system will overwrite project settings and will ignore any student's added files and will returned program to the original state

### UnitTestFixture file (if provided) needs to be set by user

- Grading will be on the UnitTestFixture settings
  - Please explicitly set which tests you want graded... no regrading if set incorrectly

## Due Dates

- ~~See Piazza for due date and time~~
- Submit program performance in your student directory assignment supplied.
- Fill out your this **Submission Report** and commit to performance
  - **ONLY** use Adobe Reader to fill out form, all others will be rejected.
  - Fill out the form and discussion for full credit.

## Goals

- Single Linked list
  - Fundamentals
- Increasing C++ knowledge and understanding

## Assignments

- Write a double linked list
  - Add to front of list
  - Add to end of list
  - Remove any node
  - Print the contents of the list
  - Sort the list
    - Suggestion (use insertion sort)
    - Any sort will do... as long as you keep it linked list
- Simple double linked list
  - No tail pointers

```
class Card
{
public:
    Card(const char *_pName, int val);
    void Print();

    // do not add data
    Card *pNext;
    Card *pPrev;
    const char *pName;
    int key;
};
```

```
class Deck
{
public:
    // do not add data
    Card *pHead;
};
```

- Have your output MATCH this sample

```
*****
**      Framework: 3.82          **
**    C++ Compiler: 193732822    **
**   Tools Version: 14.37.32822  **
**  Windows SDK: 10.0.22621.0   **
**  Mem Tracking: enabled       **
**          Mode: x86 Debug      **
*****
```

```
-----
Memory Tracking: start()
-----
```

```
Deck(Add to last (A,B,C,D,E,F)): head:A
Card(A) val:11
next-> B
prev-> null
Card(B) val:55
next-> C
prev-> A
Card(C) val:33
next-> D
prev-> B
Card(D) val:77
next-> E
prev-> C
Card(E) val:99
next-> F
prev-> D
Card(F) val:44
next-> null
prev-> E
```

```
Deck(sorted lowest to highest): head:A
Card(A) val:11
next-> C
prev-> null
Card(C) val:33
next-> F
prev-> A
Card(F) val:44
next-> B
prev-> C
Card(B) val:55
next-> D
prev-> F
Card(D) val:77
next-> E
```

```
prev-> B
Card(E) val:99
next-> null
prev-> D
```

Deck(remove A,B,C,D,E,F): head:null

Deck(Add to last (A,B,C,D,E,F)): head:F

```
Card(F) val:44
next-> E
prev-> null
Card(E) val:99
next-> D
prev-> F
Card(D) val:77
next-> C
prev-> E
Card(C) val:33
next-> B
prev-> D
Card(B) val:55
next-> A
prev-> C
Card(A) val:11
next-> null
prev-> B
```

Deck(sorted lowest to highest): head:A

```
Card(A) val:11
next-> C
prev-> null
Card(C) val:33
next-> F
prev-> A
Card(F) val:44
next-> B
prev-> C
Card(B) val:55
next-> D
prev-> F
Card(D) val:77
next-> E
prev-> B
Card(E) val:99
next-> null
prev-> D
```

Deck(remove A,B,C,D,E,F): head:null

```
-----
Memory Tracking: passed
-----
```

```
Memory Tracking: end()
-----
```

- Use the main() provided
  - If you complete this assignment and your output looks like sample
  - Submit it
    - Fill in the PDF as usual
    - Fill in the box on what PA you want regraded
      - Max 75% PA grade will be given
    - Private piazza me that your BootCamp is done
      - I can added it to the regrade list
    - BootCamp is 100% optional